

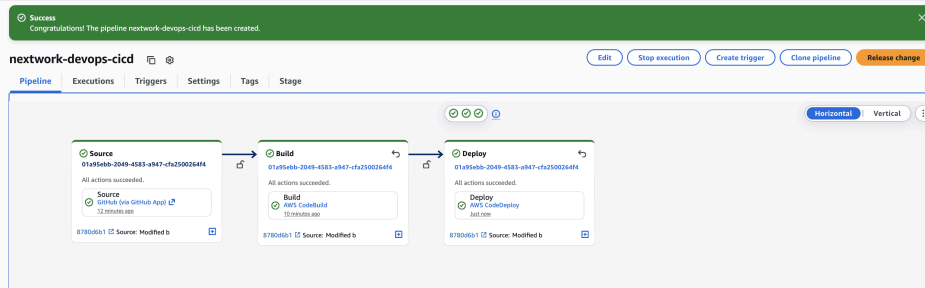


nextwork.org

Build a CI/CD Pipeline with AWS

CH

Arpita Chowdhury





Introducing Today's Project!

In this project, I built a complete CI/CD pipeline using AWS CodePipeline to connect source code, build, and deployment stages into a fully automated workflow. I tested automatic deployments by pushing code changes and observing them deploy end-to-end, then intentionally introduced a breaking change to see how the pipeline fails and how rollbacks/recovery work. I did this project to understand how real cloud engineering teams automate releases, reduce manual errors, and safely recover from failed deployments.

Key tools and concepts

Services I used were AWS CodePipeline to orchestrate the end-to-end CI/CD workflow, GitHub as the source code repository, AWS CodeBuild to compile and package the application into a deployable artifact, Amazon S3 to store build artifacts, AWS CodeDeploy to automate deployments and rollbacks to EC2, AWS EC2 to host the web application, IAM for role-based access control, and AWS CloudFormation to provision supporting infrastructure. Key concepts I learnt include CI/CD pipeline design, webhook-based triggers, artifact flow between stages, IAM roles and least-privilege permissions, CodeDeploy agent behavior, EC2 tagging for deployment groups, handling deployment failures, manual and automatic rollbacks, and how to verify production changes end-to-end by testing real code updates.

Project reflection

This project took me approximately. 4 hours.



The most challenging part was debugging IAM permissions, deployment failures, and environment configuration issues across multiple AWS services, especially when a single missing permission could cause the entire pipeline to fail. It was most rewarding to see CodePipeline automatically trigger from a GitHub push, successfully deploy the application, and then roll back cleanly to a stable version, proving that the full CI/CD and disaster-recovery workflow was working exactly as it would in a real production environment.

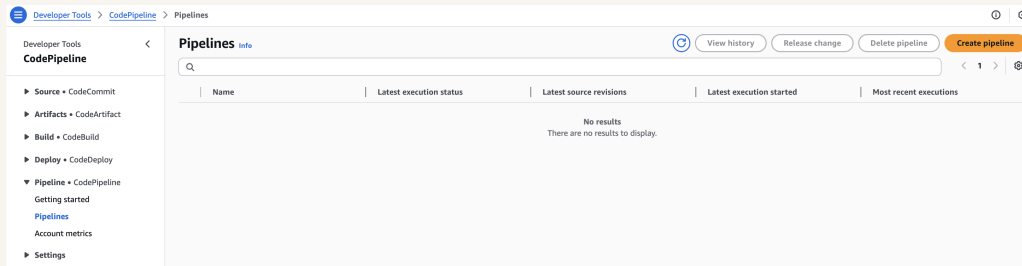


Starting a CI/CD Pipeline

AWS CodePipeline is a fully managed continuous integration and continuous delivery (CI/CD) service that automates the process of pulling source code changes, building and testing the application, and deploying it to production or staging environments, enabling teams to deliver updates faster, more reliably, and with consistent, repeatable workflows.

CodePipeline offers different execution modes based on how and when pipeline runs are triggered and controlled. I chose Suspended so the pipeline does not automatically execute while I am still configuring and validating all stages, whereas other options include Queued, which runs executions in order, and Parallel, which allows multiple executions to run at the same time when changes are pushed.

A service role gets created automatically during setup so AWS CodePipeline can securely access and coordinate other AWS services like CodeBuild, CodeDeploy, S3, and CodeArtifact on my behalf, without using my personal credentials, ensuring proper permission management and following AWS security best practices.

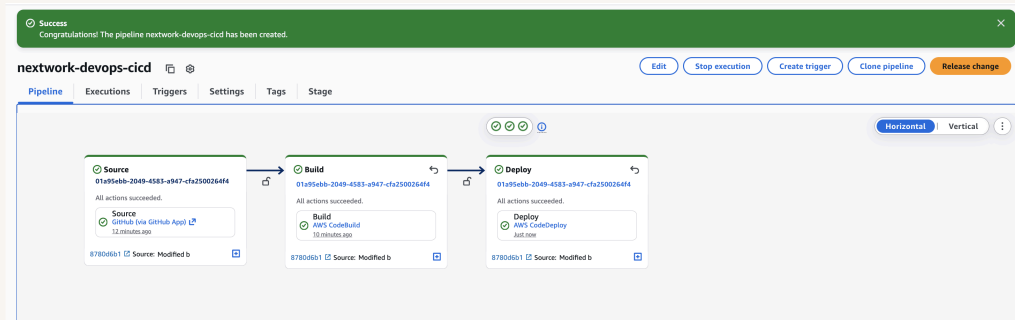




CI/CD Stages

The three stages I've set up in my CI/CD pipeline are Source, Build, and Deploy. While setting up the Source stage, I learned how CodePipeline connects to GitHub using a secure connection and webhooks to automatically trigger the pipeline whenever code is pushed to the selected branch. During the Build stage, I learned how CodeBuild compiles the application, installs dependencies, and packages the output as a deployable artifact using a `buildspec.yml`. Finally, while configuring the Deploy stage, I learned how CodeDeploy uses deployment groups, instance tags, and the CodeDeploy agent on EC2 to reliably deliver the built artifact to the target instance, handle lifecycle events, and manage deployment failures and retries.

CodePipeline organizes the three stages into a visual, sequential workflow where Source, Build, and Deploy run in order and clearly show the flow of code from GitHub to production. In each stage, you can click the stage or action to see execution details, including the commit ID from GitHub, CodeBuild logs and `buildspec` output, and CodeDeploy deployment status and lifecycle events, which makes it easy to track progress, troubleshoot failures, and verify that every step of the CI/CD process completed successfully





Source Stage

In the Source stage, the default branch tells CodePipeline which GitHub branch to pull code from when the pipeline is triggered manually or from a non-branch-specific event, ensuring the pipeline always uses a consistent and expected version of the source code for builds and deployments.

The source stage is also where you enable webhook events, which allow CodePipeline to automatically start a new pipeline execution whenever a change (such as a push or merge) is made to the connected GitHub repository and branch, enabling real-time, event-driven CI/CD instead of manual or scheduled runs.

Step 1
[Choose creation option](#)

Step 2
[Choose pipeline settings](#)

Step 3
Add source stage

Step 4 - optional
[Add build stage](#)

Step 5 - optional
[Add test stage](#)

Step 6
[Add deploy stage](#)

Step 7
[Review](#)

Add source stage [Info](#)

Step 3 of 7

Source

Source provider
This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.

GitHub (via GitHub App) ▼

Connection
Choose an existing connection that you have already configured, or create a new one and then return to this task.

or [Connect to GitHub](#)

Repository name
Choose a repository in your GitHub account.

You can type or paste the group path to any project that the provided credentials can access. Use the format 'group/subgroup/project'.

Default branch
Default branch will be used only when pipeline execution starts from a different source or manually started.

Output artifact format
Choose the output artifact format.

☒ **CodePipeline default**
AWS CodePipeline uses the default zip format for artifacts in the pipeline. Does not include Git metadata about the repository.

☐ **Full clone**
AWS CodePipeline passes metadata about the repository that allows subsequent actions to do a full Git clone. Only supported for AWS CodeBuild actions. [Learn more](#)

☒ **Enable automatic retry on stage failure**



Build Stage

The Build stage uses AWS CodeBuild to process the SourceArtifact from the GitHub source stage, run a single automated build in the us-east-1 region, and prepare deployable artifacts for the next stage in the pipeline.

Environment variables - optional
Choose the key, value, and type for your CodeBuild environment variables. In the value field, you can reference variables generated by CodePipeline. [Learn more](#)

[Add environment variable](#)

Build type

☒ **Single build**
Triggers a single build.

☐ **Batch build**
Triggers multiple builds as a single execution.

Region

United States (N. Virginia) ▼

Input artifacts
Choose an input artifact for this action. [Learn more](#)

SourceArtifact ×
Defined by: Source

☒ Enable automatic retry on stage failure

[Cancel](#) [Previous](#) [Skip build stage](#) [Next](#)



Deploy Stage

The Deploy stage is where the pipeline takes the final build artifact and deploys it to the EC2 instance using AWS CodeDeploy. In this stage, I configured CodeDeploy to use the nextwork-devops-cicd application and the nextwork-devops-cicd-deployment-group, which targets the EC2 instance based on tags and manages the deployment lifecycle. CodeDeploy pulls the packaged artifact produced by the build stage from Amazon S3, follows the instructions defined in appspec.yml, and executes the required lifecycle events to safely stop the existing application, install the new version, and bring the service back online.

The screenshot shows the 'Deploy - optional' configuration screen in the AWS CodeDeploy console. The form is set up with the following details:

- Deploy provider:** AWS CodeDeploy
- Region:** United States (N. Virginia)
- Input artifacts:** BuildArtifact (Defined by: Build)
- Application name:** nextwork-devops-cicd
- Deployment group:** nextwork-devops-cicd-deployment-group
- Configuration options:**
 - ☒ Configure automatic rollback on stage failure
 - ☐ Enable automatic retry on stage failure

At the bottom of the form, there are four buttons: 'Cancel', 'Previous', 'Skip deployment stage', and 'Next'.



Success!

Since my CI/CD pipeline gets triggered by GitHub webhook events on pushes to my configured branch, it automatically starts a new execution whenever I commit and push code changes to the repository. I tested my pipeline by making a small update in the project, committing it, and pushing it to GitHub, then monitoring CodePipeline as it pulled the latest commit in the Source stage, ran the build in CodeBuild, and deployed the updated artifact to my EC2 instance through CodeDeploy to confirm the end-to-end automation worked.

The moment I pushed the code change to GitHub, the webhook instantly triggered a new CodePipeline execution, and I could see the Source, Build, and Deploy stages start running automatically without any manual action. The commit message shown under each stage reflects the exact Git commit that the pipeline is processing, which helps confirm that all stages are using the same version of the code and makes it easy to trace each build and deployment back to a specific change in the repository.

Once my pipeline executed successfully, I checked the live web application running on the EC2 instance, and the newly added line appeared exactly as expected on the page. This confirmed that the code change was pulled from GitHub, built by CodeBuild, and automatically deployed to production by CodeDeploy through CodePipeline, proving that the end-to-end CI/CD workflow is working correctly with no manual intervention.



Arpita Chowdhury

NextWork Student

nextwork.org

Hello NextWork!

This is my NextWork web application working!

If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :D

If you see this line, that means your latest changes are automatically deployed into production by CodePipeline!



Testing the Pipeline

In this project extension, I initiated a manual rollback on the CodeDeploy deploy stage through CodePipeline, selecting a previously successful deployment revision to restore the application. Once the rollback completed, I verified that the web application reverted to its earlier state, confirming that the last stable version was redeployed correctly. Automatic rollback is important because it minimizes downtime and risk by quickly restoring a known-good version when a deployment fails, ensuring application reliability and demonstrating strong disaster-recovery and production-readiness practices in a real-world DevOps environment.

During the rollback, the Source and Build stages are not re-run because CodePipeline reuses a previously built and approved artifact instead of pulling new code or rebuilding it. I could verify this by comparing the commit ID and artifact revision shown in the Deploy stage with an earlier successful execution, and by observing that the Source and Build stages did not trigger again while the application content on the EC2 instance reverted to the earlier version.

After rollback, the live web app reverted to the previous stable version of the application, where the most recent CodePipeline-deployed line was no longer visible and the page content matched the earlier deployment state. This confirmed that CodeDeploy successfully restored the last known good artifact, demonstrating that the rollback correctly returned the application to a healthy and working version without requiring a new build.



Arpita Chowdhury

NextWork Student

nextwork.org

✔ **Deploy**

Rollback

Succeeded

Start rollback

✕

Deploy

[AWS CodeDeploy](#)

✔ Succeeded - 2 minutes ago



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

